

Reinforcement Learning

Vanilla Policy Gradient & REINFORCE

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 4, 2026

1. Recap
2. Learning Outcomes
3. Motivation for Policy Gradient Methods
4. Policy Gradients
5. REINFORCE
6. Variance Reduction
7. On-Policy vs. Off-Policy Learning
8. REINFORCE in action: Recurrent Attention Model (RAM)
9. Policy Gradients in Action: AlphaGo Zero
10. Value-Based and Policy-Based RL
11. Policy Gradient vs. Q-Learning
12. Summary
13. References

- ▶ A Markov Decision Process (MDP) defines the RL problem using:
 - States $\mathcal{S}$
 - Actions $\mathcal{A}$
 - Rewards $\mathcal{R}$
 - Transition probabilities $\mathbb{P}$
 - Discount factor γ
- ▶ Markov property: The future depends only on the present state, not the past.
- ▶ Agent and environment interact in a loop.
- ▶ Policy π decides the agent's actions.

- ▶ Value function: Measures how good a state is.
- ▶ Q-value function: Measures how good a state-action pair is.
- ▶ Bellman equation: Recursively defines value and Q-value functions.
- ▶ Q-learning: Updates Q-values to reduce Bellman error.
- ▶ Deep Q-Learning: Uses neural networks to approximate Q-values.
- ▶ SARSA: On-policy version of Q-learning (learns from actions the policy actually takes).
- ▶ Deterministic vs. Stochastic policy: $\pi(s) = a$ vs. $\pi(a|s) = \Pr(a_t = a \mid s_t = s)$.

- Advantage function: $A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s)$ — how much better an action is than average in that state.

By the end of this session, you will be able to:

- ▶ Understand the **limitations of value-based methods** like Q-learning.
- ▶ Formally define and derive the **Policy Gradient** objective.
- ▶ Implement and interpret the **REINFORCE algorithm**.
- ▶ Diagnose the **high-variance** problem and apply **variance-reduction** techniques — causality, discounting, baselines, and the advantage function.
- ▶ Explain the **on-policy vs. off-policy** distinction and its consequences for sample efficiency.
- ▶ Evaluate policy-based methods in different environments.

- ▶ Recall from RL Foundations: RL methods are either **value-based** or **policy-based**.
- ▶ So far we have covered the value-based branch: Q-learning, DQN, SARSA.
- ▶ Today we move to the other branch: **directly optimizing the policy**.

- ▶ Recall from RL Foundations: RL methods are either **value-based** or **policy-based**.
- ▶ So far we have covered the value-based branch: Q-learning, DQN, SARSA.
- ▶ Today we move to the other branch: **directly optimizing the policy**.
- ▶ Value-based methods learn Q-values and derive the policy indirectly.
 - Inefficient in continuous or large action spaces
 - Can't represent stochastic policies
 - May lead to high variance and instability

Policy Gradient Methods:

- Learn policy parameters directly to maximize expected return:

$$\mathcal{J}(\theta) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{t \geq 0} \gamma^t r_t \right]$$

Reinforcement Learning: **Policy Gradients**

- ▶ To act well, you don't need the exact worth of every option — you only need to pick the best one.
- ▶ $Q(s, a)$ answers the **hard** question: *"How much total future reward will I get for every action, in every state?"* — a detailed, complicated function of the whole world.
- ▶ $\pi(s)$ answers the **easy** question: *"What should I do right now?"* — often a simple rule.

- ▶ **To act well, you don't need the exact worth of every option — you only need to pick the best one.**
 - ▶ $Q(s, a)$ answers the **hard** question: *“How much total future reward will I get for every action, in every state?”* — a detailed, complicated function of the whole world.
 - ▶ $\pi(s)$ answers the **easy** question: *“What should I do right now?”* — often a simple rule.
 - ▶ **Grasping:** figuring out the exact long-term payoff of every possible gripper motion is hard. But the actual behavior — “close the hand once it's around the object” — is simple.
- ⇒ So why solve the hard problem (Q) just to act? **Learn the policy directly.**

- Formally, let us define a class of parameterized policies:

$$\Pi = \{\pi_{\theta} \mid \theta \in \mathbb{R}^m\}$$

- For each policy, we can define its expected return:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_{\theta} \right]$$

- Formally, let us define a class of parameterized policies:

$$\Pi = \{\pi_{\theta} \mid \theta \in \mathbb{R}^m\}$$

- For each policy, we can define its expected return:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_{\theta} \right]$$

- Our goal is to find the optimal policy: $\theta^* = \arg \max_{\theta} \mathcal{J}(\theta)$
- How can we achieve this?

- Formally, let us define a class of parameterized policies:

$$\Pi = \{\pi_{\theta} \mid \theta \in \mathbb{R}^m\}$$

- For each policy, we can define its expected return:

$$\mathcal{J}(\theta) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t r_t \mid \pi_{\theta} \right]$$

- Our goal is to find the optimal policy: $\theta^* = \arg \max_{\theta} \mathcal{J}(\theta)$
- How can we achieve this?
- **Solution:** Perform gradient ascent on the policy parameters!

- **Discrete actions** — Softmax policy:

$$\pi_\theta(a | s) = \frac{e^{\phi(s,a)^\top \theta}}{\sum_{a'} e^{\phi(s,a')^\top \theta}}$$

The network outputs one logit per action; softmax converts them to probabilities.

- **Discrete actions** — Softmax policy:

$$\pi_\theta(a | s) = \frac{e^{\phi(s,a)^\top \theta}}{\sum_{a'} e^{\phi(s,a')^\top \theta}}$$

The network outputs one logit per action; softmax converts them to probabilities.

- **Continuous actions** — Gaussian policy:

$$\pi_\theta(a | s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$$

The network outputs a mean and standard deviation; actions are sampled from the resulting Gaussian.

- **Discrete actions** — Softmax policy:

$$\pi_\theta(a | s) = \frac{e^{\phi(s,a)^\top \theta}}{\sum_{a'} e^{\phi(s,a')^\top \theta}}$$

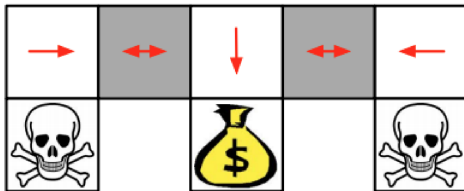
The network outputs one logit per action; softmax converts them to probabilities.

- **Continuous actions** — Gaussian policy:

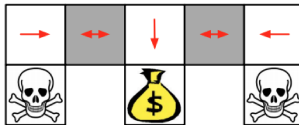
$$\pi_\theta(a | s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$$

The network outputs a mean and standard deviation; actions are sampled from the resulting Gaussian.

- The Gaussian policy will be central to continuous-control algorithms (DDPG, SAC).
- Both are differentiable w.r.t. θ , which is what makes gradient ascent possible.



- Grey states are **aliased**: the agent observes the same features in multiple distinct positions
- **Key assumption — why they look identical**: the agent perceives only *local* features, not its absolute position. The two grey cells are equidistant from the goal and the danger squares and share the same surrounding walls, so every feature it can sense is identical — it cannot tell them apart and must treat them as one state



- ▶ A **deterministic** policy must make the same choice in every grey state — always go East or always go West — so the agent gets stuck in a loop and never reaches the goal
- ▶ A **stochastic** 50/50 policy breaks the symmetry: the agent eventually reaches the goal with probability 1
- ▶ This is the concrete motivation for the **softmax** and **Gaussian** policy parameterisations introduced earlier

Reinforcement Learning: **REINFORCE**

- ▶ REINFORCE is an elegant algorithm for maximizing the expected return.
- ▶ Intuition: trial and error.
- ▶ Sample a trajectory τ . If you get a high reward, try to make it more likely; if you get a low reward, try to make it less likely.
- ▶ A trajectory is a sequence of states, actions, and rewards: $\tau = (s_0, a_0, r_0, s_1, a_1, \dots)$.

Our objective is the **expected** discounted return, where $r(\tau) := \sum_{t \geq 0} \gamma^t r_t$ is the return of one trajectory:

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(\tau)] = \int_{\tau} r(\tau) p(\tau; \theta) d\tau$$

Reminder: an expectation is a probability-weighted average

$\mathbb{E}_{x \sim p}[f(x)] = \sum_x \underbrace{p(x)}_{\text{how likely}} \underbrace{f(x)}_{\text{value}}$ (or $\int p(x)f(x) dx$ if continuous). Here each “outcome” is a whole trajectory τ : its value is $r(\tau)$ and its weight is how likely the policy is to produce it.

Our objective is the **expected** discounted return, where $r(\tau) := \sum_{t \geq 0} \gamma^t r_t$ is the return of one trajectory:

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim p(\tau; \theta)} [r(\tau)] = \int_{\tau} r(\tau) p(\tau; \theta) d\tau$$

Differentiate w.r.t. θ . The gradient moves inside the integral, and $r(\tau)$ carries no θ (the rewards of a *fixed* trajectory don't change with the weights), so only $p(\tau; \theta)$ is differentiated:

$$\nabla_{\theta} \mathcal{J}(\theta) = \int_{\tau} r(\tau) \nabla_{\theta} p(\tau; \theta) d\tau$$

where the probability of a trajectory factorises as

$$p(\tau; \theta) = \underbrace{p(s_0)}_{\text{start state (no } \theta)} \prod_{t \geq 0} \underbrace{p(s_{t+1} | s_t, a_t)}_{\text{environment (no } \theta)} \underbrace{\pi_{\theta}(a_t | s_t)}_{\text{our policy}}$$

This gradient is **intractable** — for two distinct reasons that the rest of the derivation will fix one at a time.

Problem 1 — we have no way to actually compute this

The integral runs over *every possible trajectory* — far too many to add up directly. Our only tool for such a sum is **Monte Carlo**: run the policy and average what happens. But that works only when each trajectory is weighted by a genuine probability $p(\tau; \theta)$ (non-negative, sums to 1) — then “sampling” means playing episodes. Here the weight is $\nabla_{\theta} p(\tau; \theta)$, a *change* in probability, not a probability: it can be negative and it sums to 0. You cannot play an episode to “sample” a change, so Monte Carlo does not apply.

This gradient is **intractable** — for two distinct reasons that the rest of the derivation will fix one at a time.

Problem 1 — we have no way to actually compute this

The integral runs over *every possible trajectory* — far too many to add up directly. Our only tool for such a sum is **Monte Carlo**: run the policy and average what happens. But that works only when each trajectory is weighted by a genuine probability $p(\tau; \theta)$ (non-negative, sums to 1) — then “sampling” means playing episodes. Here the weight is $\nabla_{\theta} p(\tau; \theta)$, a *change* in probability, not a probability: it can be negative and it sums to 0. You cannot play an episode to “sample” a change, so Monte Carlo does not apply.

Problem 2 — it needs a model of the environment

$p(\tau; \theta)$ contains $p(s_0)$ and $p(s_{t+1}|s_t, a_t)$: the environment's dynamics, which we **do not know** and cannot differentiate.

Goal: rewrite the gradient as an expectation $\mathbb{E}_{\tau \sim p(\tau; \theta)}[\dots]$ — the *only* form we can estimate, by rolling out the policy and averaging.

Goal: rewrite the gradient as an expectation $\mathbb{E}_{\tau \sim p(\tau; \theta)}[\dots]$ — the *only* form we can estimate, by rolling out the policy and averaging.

The log-derivative trick — multiply and divide by $p(\tau; \theta)$:

$$\nabla_{\theta} p(\tau; \theta) = p(\tau; \theta) \frac{\nabla_{\theta} p(\tau; \theta)}{p(\tau; \theta)} = p(\tau; \theta) \nabla_{\theta} \log p(\tau; \theta)$$

This manufactures a factor of $p(\tau; \theta)$ — exactly the sampling weight an expectation needs.

Goal: rewrite the gradient as an expectation $\mathbb{E}_{\tau \sim p(\tau; \theta)}[\cdots]$ — the *only* form we can estimate, by rolling out the policy and averaging.

The log-derivative trick — multiply and divide by $p(\tau; \theta)$:

$$\nabla_{\theta} p(\tau; \theta) = p(\tau; \theta) \frac{\nabla_{\theta} p(\tau; \theta)}{p(\tau; \theta)} = p(\tau; \theta) \nabla_{\theta} \log p(\tau; \theta)$$

This manufactures a factor of $p(\tau; \theta)$ — exactly the sampling weight an expectation needs.

Substitute it back into $\nabla_{\theta} \mathcal{J}$:

$$\nabla_{\theta} \mathcal{J}(\theta) = \int_{\tau} (r(\tau) \nabla_{\theta} \log p(\tau; \theta)) p(\tau; \theta) d\tau = \mathbb{E}_{\tau \sim p(\tau; \theta)}[r(\tau) \nabla_{\theta} \log p(\tau; \theta)]$$

It *is* an expectation now $\Rightarrow$ estimate it by Monte Carlo. **Problem 1 solved** (Problem 2 still hides inside $\log p$).

Goal: the log-probability $\log p(\tau; \theta)$ still contains the unknown environment terms. Watch them vanish under ∇_{θ} .

Recall the trajectory probability is a **product** over timesteps:

$$p(\tau; \theta) = p(s_0) \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

Goal: the log-probability $\log p(\tau; \theta)$ still contains the unknown environment terms. Watch them vanish under ∇_{θ} .

Recall the trajectory probability is a **product** over timesteps:

$$p(\tau; \theta) = p(s_0) \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

Taking log turns this product into a **sum** (and since log is increasing, maximizing $\log p$ is the same as maximizing p):

$$\log p(\tau; \theta) = \log p(s_0) + \sum_{t \geq 0} \left[\log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t) \right]$$

Goal: the log-probability $\log p(\tau; \theta)$ still contains the unknown environment terms. Watch them vanish under ∇_{θ} .

Recall the trajectory probability is a **product** over timesteps:

$$p(\tau; \theta) = p(s_0) \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

Taking log turns this product into a **sum** (and since log is increasing, maximizing $\log p$ is the same as maximizing p):

$$\log p(\tau; \theta) = \log p(s_0) + \sum_{t \geq 0} \left[\log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t) \right]$$

Differentiate. Every term without θ becomes zero — the start state $\log p(s_0)$ and *all* dynamics $\log p(s_{t+1} | s_t, a_t)$:

$$\nabla_{\theta} \log p(\tau; \theta) = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Goal: the log-probability $\log p(\tau; \theta)$ still contains the unknown environment terms. Watch them vanish under ∇_{θ} .

Recall the trajectory probability is a **product** over timesteps:

$$p(\tau; \theta) = p(s_0) \prod_{t \geq 0} p(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

Taking log turns this product into a **sum** (and since log is increasing, maximizing $\log p$ is the same as maximizing p):

$$\log p(\tau; \theta) = \log p(s_0) + \sum_{t \geq 0} \left[\log p(s_{t+1} | s_t, a_t) + \log \pi_{\theta}(a_t | s_t) \right]$$

Differentiate. Every term without θ becomes zero — the start state $\log p(s_0)$ and *all* dynamics $\log p(s_{t+1} | s_t, a_t)$:

$$\nabla_{\theta} \log p(\tau; \theta) = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

Only our own policy survives $\Rightarrow$ the estimator is **model-free**. **Problem 2 solved.**

Put both fixes together: start from the expectation (Problem 1 fix) and substitute $\nabla_{\theta} \log p = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ (Problem 2 fix):

$$\begin{aligned}\nabla_{\theta} \mathcal{J}(\theta) &= \mathbb{E}_{\tau}[r(\tau) \nabla_{\theta} \log p(\tau; \theta)] \\ &= \mathbb{E}_{\tau} \left[r(\tau) \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \approx \sum_{t \geq 0} r(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)\end{aligned}$$

(Last step: one sampled trajectory τ . Here $r(\tau)$ is constant over t , so it could equally sit outside the sum — we keep it inside because the next section replaces it with a *per-timestep* weight.)

Put both fixes together: start from the expectation (Problem 1 fix) and substitute $\nabla_{\theta} \log p = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ (Problem 2 fix):

$$\begin{aligned}\nabla_{\theta} \mathcal{J}(\theta) &= \mathbb{E}_{\tau} [r(\tau) \nabla_{\theta} \log p(\tau; \theta)] \\ &= \mathbb{E}_{\tau} \left[r(\tau) \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \approx \sum_{t \geq 0} r(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)\end{aligned}$$

(Last step: one sampled trajectory τ . Here $r(\tau)$ is constant over t , so it could equally sit outside the sum — we keep it inside because the next section replaces it with a *per-timestep* weight.)

Interpretation: $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ points in the direction that makes a_t *more likely*; the scalar $r(\tau)$ signs and scales that push:

- ▶ $r(\tau)$ high $\Rightarrow$ increase the probability of the actions taken;
- ▶ $r(\tau)$ low / negative $\Rightarrow$ decrease them.

Put both fixes together: start from the expectation (Problem 1 fix) and substitute $\nabla_{\theta} \log p = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ (Problem 2 fix):

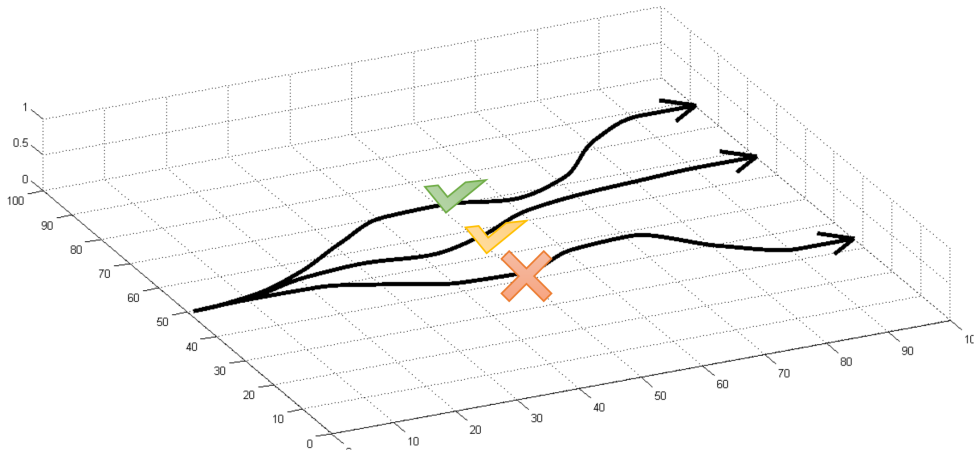
$$\begin{aligned}\nabla_{\theta} \mathcal{J}(\theta) &= \mathbb{E}_{\tau} [r(\tau) \nabla_{\theta} \log p(\tau; \theta)] \\ &= \mathbb{E}_{\tau} \left[r(\tau) \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right] \approx \sum_{t \geq 0} r(\tau) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)\end{aligned}$$

(Last step: one sampled trajectory τ . Here $r(\tau)$ is constant over t , so it could equally sit outside the sum — we keep it inside because the next section replaces it with a *per-timestep* weight.)

Interpretation: $\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$ points in the direction that makes a_t *more likely*; the scalar $r(\tau)$ signs and scales that push:

- ▶ $r(\tau)$ high $\Rightarrow$ increase the probability of the actions taken;
- ▶ $r(\tau)$ low / negative $\Rightarrow$ decrease them.

Crediting every action with the *whole*-trajectory return looks crude — but it is **unbiased**: in expectation, it averages out.



Reinforcement Learning: **Variance Reduction**

Our gradient is estimated from *sampled* trajectories, so it is never exact. There are two independent ways such an estimate can be wrong:

Variance — how much it bounces

Run the same policy again and you get different trajectories, hence a different gradient estimate. **High variance** = the estimate jumps around from one batch to the next $\Rightarrow$ you need many samples to average out the noise $\Rightarrow$ slow, unstable learning.

Bias — whether it points the right way on average

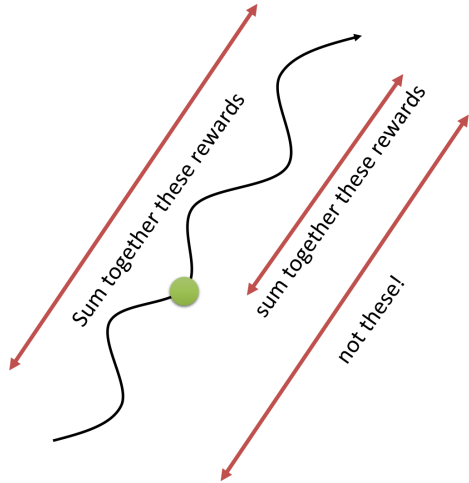
Unbiased = averaged over infinitely many samples it equals the true gradient $\nabla_{\theta} \mathcal{J}$. **Biased** = systematically off, converging toward a slightly wrong direction *even with infinite data*.

Dartboard picture: **variance** is how scattered the darts land; **bias** is whether they cluster off the bullseye. *The whole game below*: accept a little bias if it buys a large drop in variance — the net result learns faster.

- ▶ However, there is a problem.
- ▶ This approach suffers from high variance because credit assignment is difficult.
- ▶ Can we help the estimator?

- ▶ However, there is a problem.
- ▶ This approach suffers from high variance because credit assignment is difficult.
- ▶ Can we help the estimator?
- ▶ **First idea — the Causality Trick:** Rewards before time t cannot be affected by action a_t , so they carry no useful gradient signal. Use only future rewards:

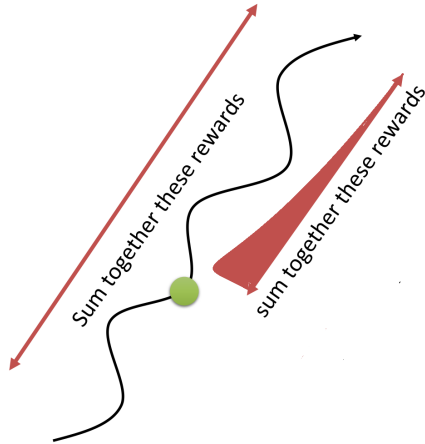
$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} r_{t'} \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$



- ▶ The causality trick is **unbiased** — removing past rewards is mathematically correct, so on average the gradient is unchanged.
- ▶ But the reward-to-go $\sum_{t' \geq t} r_{t'}$ is still a sum over an *entire* future rollout, driven by many random action choices and random environment transitions. Run the **same** policy twice and this number can come out wildly different.
- ▶ That swing is the **high variance**: the gradient estimate bounces from sample to sample, so it takes a huge number of trajectories to average out — learning is slow and unstable.

- ▶ The causality trick is **unbiased** — removing past rewards is mathematically correct, so on average the gradient is unchanged.
- ▶ But the reward-to-go $\sum_{t' \geq t} r_{t'}$ is still a sum over an *entire* future rollout, driven by many random action choices and random environment transitions. Run the **same** policy twice and this number can come out wildly different.
- ▶ That swing is the **high variance**: the gradient estimate bounces from sample to sample, so it takes a huge number of trajectories to average out — learning is slow and unstable.
- ▶ **Second idea — discounting**: weight future rewards by $\gamma^{t'-t}$, so distant, largely-unrelated rewards count for less. This trades a small **bias** for a large drop in variance:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$



- **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.

- ▶ **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.
- ▶ **What is important then?** Whether a reward is better or worse than what you expect to get.

- ▶ **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.
- ▶ **What is important then?** Whether a reward is better or worse than what you expect to get.
- ▶ **Idea:** Introduce a baseline function dependent on the state:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} - b(s_t) \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- ▶ **Problem:** The raw value of a trajectory isn't necessarily meaningful. For example, if all rewards are positive, you keep increasing the probabilities of actions.
- ▶ **What is important then?** Whether a reward is better or worse than what you expect to get.
- ▶ **Idea:** Introduce a baseline function dependent on the state:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} \left(\sum_{t' \geq t} \gamma^{t'-t} r_{t'} - b(s_t) \right) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- ▶ A simple baseline: the moving average of rewards experienced so far from all trajectories.

- ▶ Can we choose a better baseline?
- ▶ Essentially, we want to increase the probability of an action from a state if this action was better than the **expected value** from that state.

- ▶ Can we choose a better baseline?
- ▶ Essentially, we want to increase the probability of an action from a state if this action was better than the **expected value** from that state.
- ▶ What does this remind you of?

- ▶ Can we choose a better baseline?
- ▶ Essentially, we want to increase the probability of an action from a state if this action was better than the **expected value** from that state.
- ▶ What does this remind you of?
- ▶ **Recall from RL Foundations:** We defined the **Advantage function**:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s)$$

- ▶ $A^{\pi}(s_t, a_t) > 0$: action a_t was better than average — increase its probability.
- ▶ $A^{\pi}(s_t, a_t) < 0$: action a_t was worse than average — decrease its probability.

- ▶ Using the advantage as our baseline gives us the estimator:

$$\nabla_{\theta} \mathcal{J}(\theta) \approx \sum_{t \geq 0} A^{\pi}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$$

- ▶ This is the best possible baseline: it directly measures how much better an action was than the policy's own average.
- ▶ It is unbiased, and it provides lower variance than a fixed moving-average baseline.

*But how do we actually estimate A^{π} ? That — learning a **critic** — is exactly what Actor-Critic methods are about.*

Initialize policy network π_θ with random weights θ ; set step size α and discount γ

for $episode = 1$ **to** M **do**

 Run π_θ for one full episode of length T , producing

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, r_{T-1})$$

for $t = 0$ **to** $T - 1$ **do**

 | Reward-to-go from step t : $R_t = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$

end

 Gradient *ascent* update (reward-to-go minus baseline $b(s_t)$):

$$\theta \leftarrow \theta + \alpha \sum_{t=0}^{T-1} (R_t - b(s_t)) \nabla_\theta \log \pi_\theta(a_t | s_t)$$

end

Algorithm: REINFORCE (Monte-Carlo Policy Gradient)

⁰Williams (1992), *Simple statistical gradient-following algorithms for connectionist reinforcement learning*, *Machine Learning* 8. An episode of length T has actions $a_0, \dots, a_{T-1}$ (T in total); the terminal state s_T has no action, so the sum stops at $T - 1$. The baseline $b(s_t)$ is optional — $b \equiv 0$ gives vanilla REINFORCE, and a running average of returns is a simple choice. We *ascend* because we maximize expected return $\mathcal{J}(\theta)$, whereas DQN minimized an error (descent).

Reinforcement Learning: **On-Policy** vs. **Off-Policy**

- ▶ Recall from value-based methods: SARSA is **on-policy** — it updates using the action the current policy actually takes.
- ▶ Q-learning is **off-policy** — it updates using $\max_{a'} Q(s', a')$, regardless of what the policy does.

- ▶ Recall from value-based methods: SARSA is **on-policy** — it updates using the action the current policy actually takes.
- ▶ Q-learning is **off-policy** — it updates using $\max_{a'} Q(s', a')$, regardless of what the policy does.
- ▶ **REINFORCE is also on-policy**: it samples full trajectories under the current policy π_θ and uses them to estimate the gradient.
- ▶ Once the policy parameters θ are updated, the old trajectories are discarded — they came from a different policy and cannot be reused.

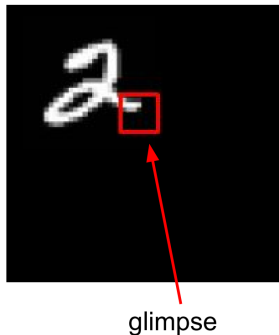
- ▶ Recall from value-based methods: SARSA is **on-policy** — it updates using the action the current policy actually takes.
- ▶ Q-learning is **off-policy** — it updates using $\max_{a'} Q(s', a')$, regardless of what the policy does.
- ▶ **REINFORCE is also on-policy**: it samples full trajectories under the current policy π_θ and uses them to estimate the gradient.
- ▶ Once the policy parameters θ are updated, the old trajectories are discarded — they came from a different policy and cannot be reused.
- ▶ **Consequence**: REINFORCE cannot use an experience replay buffer.
 - Replay stores transitions from past policies — that is off-policy data.
 - Mixing off-policy data into an on-policy gradient estimate introduces bias.

- ▶ Recall from value-based methods: SARSA is **on-policy** — it updates using the action the current policy actually takes.
- ▶ Q-learning is **off-policy** — it updates using $\max_{a'} Q(s', a')$, regardless of what the policy does.
- ▶ **REINFORCE is also on-policy**: it samples full trajectories under the current policy π_θ and uses them to estimate the gradient.
- ▶ Once the policy parameters θ are updated, the old trajectories are discarded — they came from a different policy and cannot be reused.
- ▶ **Consequence**: REINFORCE cannot use an experience replay buffer.
 - Replay stores transitions from past policies — that is off-policy data.
 - Mixing off-policy data into an on-policy gradient estimate introduces bias.
- ▶ This sample inefficiency is a core limitation of vanilla policy gradient methods and a key motivation for the algorithms we cover later (Actor-Critic/PPO and SAC).

REINFORCE in Action: **Recurrent Attention Model (RAM)**

- ▶ **Objective:** Image classification
- ▶ The model takes a sequence of “glimpses,” selectively focusing on regions of the image to predict the class.
 - Inspired by human perception and eye movements
 - Saves computational resources $\Rightarrow$ improves scalability
 - Can ignore clutter or irrelevant parts of the image
- ▶ **State:** Glimpses observed so far
- ▶ **Action:** (x, y) coordinates (center of the next glimpse) indicating where to look next in the image
- ▶ **Reward:** 1 at the final timestep if the image is correctly classified, 0 otherwise

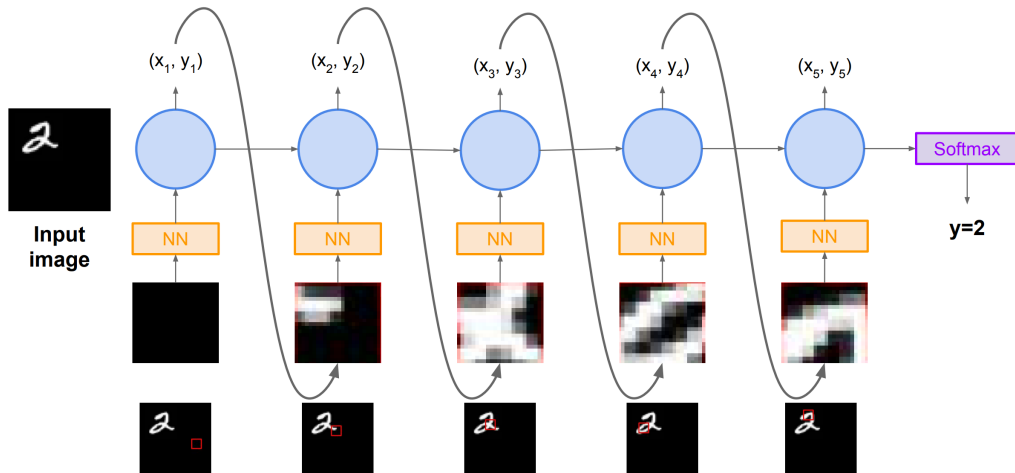
⁰[Mnih et al., 2014]



- ▶ Glimpsing is a non-differentiable operation.
- ▶ The policy for selecting glimpse locations is learned using REINFORCE.
- ▶ Given the sequence of glimpses observed so far, an RNN models the state and outputs the next action.

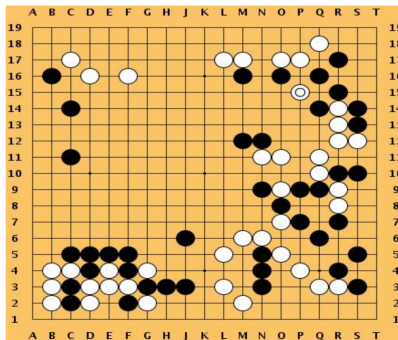
⁰[Mnih et al., 2014]

REINFORCE in Action: Recurrent Attention Model (RAM)



⁰[Mnih et al., 2014]

Policy Gradients in Action: **AlphaGo Zero**



- ▶ AlphaGo Zero (2017) reaches superhuman Go play with **no human data** — pure self-play using policy gradients
- ▶ A cleaner demonstration of REINFORCE-style learning than the original AlphaGo, which relied on supervised pretraining from human games

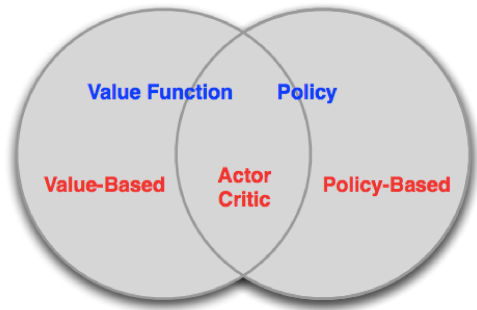
⁰Silver et al., *Mastering the game of Go without human knowledge*, Nature 550, 354–359 (2017)

- ▶ **No human games:** policy network initialised randomly; improves entirely through self-play
- ▶ **Policy gradient training:** actions sampled from the policy; wins (+1) and losses (−1) provide the reward signal
- ▶ **Value network:** learns to estimate board outcome from any position, acting as the critic
- ▶ **Monte Carlo Tree Search:** combines policy and value networks for lookahead at inference time
- ▶ Surpassed the original AlphaGo within 40 days of training

⁰Silver et al., *Mastering the game of Go without human knowledge*, Nature 550, 354–359 (2017)

Reinforcement Learning: **Value-Based** and **Policy-Based** Methods

- ▶ **Value-Based Methods**
 - Learn a value function
 - Derive policy implicitly (e.g., ϵ -greedy)
- ▶ **Policy-Based Methods**
 - Do not learn a value function
 - Learn the policy directly
- ▶ **Actor-Critic Methods** (*coming next*)
 - Learn both a value function and a policy
 - Combines the strengths of both approaches above



- ▶ Policy gradient and Q-learning use two fundamentally different representations: policies and value functions.
- ▶ Advantage of both methods: no need to model the environment.

- ▶ Policy gradient and Q-learning use two fundamentally different representations: policies and value functions.
- ▶ Advantage of both methods: no need to model the environment.
- ▶ **Policy Gradient: Pros and Cons**
 - **Pros:** Unbiased estimate of the gradient of expected return.
 - Can handle large action spaces (since only one action needs to be sampled).
 - **Cons:** High variance updates (leads to poor sample efficiency).
 - Does not perform credit assignment effectively.

- ▶ Policy gradient and Q-learning use two fundamentally different representations: policies and value functions.
- ▶ Advantage of both methods: no need to model the environment.
- ▶ **Policy Gradient: Pros and Cons**
 - **Pros:** Unbiased estimate of the gradient of expected return.
 - Can handle large action spaces (since only one action needs to be sampled).
 - **Cons:** High variance updates (leads to poor sample efficiency).
 - Does not perform credit assignment effectively.
- ▶ **Q-Learning: Pros and Cons**
 - **Pros:** Lower variance updates, more sample efficient.
 - Performs credit assignment.
 - **Cons:** Biased updates due to function approximation.
 - Difficult to handle large action spaces (since the maximum over actions must be computed).

Reinforcement Learning: **Summary**

Method	Policy Type	Gradient Source	Stability	Efficiency
Q-learning	Deterministic	Value gradients	Medium	High
REINFORCE	Stochastic	Monte Carlo returns	Low	Low
Actor-Critic	Stochastic	TD residual	Medium	Medium

- ▶ **High variance** in gradient estimates — Monte Carlo returns are noisy.
- ▶ **Sample inefficiency** — on-policy: each trajectory is used once and discarded.
- ▶ **Slow convergence** — many samples needed before the signal-to-noise ratio is acceptable.
- ▶ **No credit assignment** across timesteps beyond the causality trick and discounting.
- ▶ Variance reduction techniques (baselines, advantage) help but do not fully solve these issues.

- ▶ **Actor-Critic & PPO:** Combine a learned critic V_ϕ with the policy gradient to reduce variance. Build up to A2C, TRPO, and PPO.
- ▶ **DDPG:** Extend to deterministic policies for continuous action spaces.
- ▶ **SAC:** Add an **entropy bonus** $\mathcal{H}(\pi_\theta(\cdot|s))$ to the reward signal — encouraging exploration while maximizing return.
- ▶ **And beyond:** Exploration, model-based planning, RLHF, and open problems.

- ▶ It can be hard to learn the exact Q-value for every (state, action) pair in high-dimensional state and action spaces.
- ▶ However, we can just learn a policy that maximizes the reward.
- ▶ We can use gradient ascent on policy parameters.
- ▶ However, this can suffer from high variance. Various strategies exist to tackle this: causality, discounting, baselines, and the advantage function.
- ▶ *Next: Actor-Critic methods learn a critic V_ϕ alongside the policy to make advantage estimation practical.*

Vanilla Policy Gradient: **References**

- [1] Sutton, R. S., & Barto, A. G. (2018).
Reinforcement Learning: An Introduction (2nd ed.). MIT Press.

- [2] Williams, R. J. (1992).
Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning.
Machine Learning, 8, 229–256.

- [3] Sutton, R. S., McAllester, D. A., Singh, S. P., & Mansour, Y. (2000).
Policy Gradient Methods for Reinforcement Learning with Function Approximation.
In *Advances in Neural Information Processing Systems (NeurIPS)* 12, 1057–1063.

- [4] Silver, D., Schrittwieser, J., Simonyan, K., et al. (2017).
Mastering the Game of Go without Human Knowledge.
Nature, 550, 354–359.

- [5] Mnih, V., Heess, N., Graves, A., & Kavukcuoglu, K. (2014).
Recurrent Models of Visual Attention.
In *Advances in Neural Information Processing Systems (NeurIPS) 27*.

- [6] OpenAI Spinning Up.
<https://spinningup.openai.com>

- [7] Sergey Levine, Berkeley CS285: Deep Reinforcement Learning.
<https://rail.eecs.berkeley.edu/deeprlcourse/>

- [8] Chelsea Finn & Karol Hausman, Stanford CS224R: [Deep Reinforcement Learning](#).